Flight Booking Website — Explanation (Part 3/5)

Part 3: Backend API endpoints for flights, bookings, and messaging (World A focus).

1) Endpoint: backend/api/get-cities.php

Purpose: provide a list of distinct cities for search UI.

- Queries `flight_itinerary` to return unique city names.
- Used by search dropdowns/autocomplete on some pages.

2) Endpoint: backend/api/search-flights.php

Purpose: find flights that go from one city to another, respecting the route order.

The itinerary is stored as multiple rows in `flight_itinerary` (sequence order). Searching uses two joins: match a row for the “from” city and a row for the “to” city in the same flight, and require: `from.sequence_order < to.sequence_order`.

- **Method:** typically `GET` (with query params like `from`, `to`)
- **Output:** list of flights + sometimes company name + basic route info.

3) Endpoint: backend/api/company-flights.php

Purpose: list flights belonging to a company.

- **Input:** company id (query param)
- **DB action:** select from `flights` where `company_id` matches.
- **Extra:** joins/aggregates itinerary to display route (first city → last city).

4) Endpoint: backend/api/company-flight-details.php

Purpose: show one flight and its passengers.

- Loads flight info + company info.
- Loads pending passengers (bookings.status = pending).
- Loads registered passengers (bookings.status = registered).

This endpoint is typically used by a company dashboard “details” page.

5) Endpoint: backend/api/passenger-bookings.php

Purpose: list bookings for a passenger, including route info.

- **Input:** passenger id.
- **DB action:** select bookings joined with flights and itinerary route display.
- **Output:** a list for passenger dashboard.

6) Endpoint: backend/api/bookings.php

This file typically acts like a small controller with multiple behaviors: read bookings, create booking, cancel booking.

6.1 Create booking

- Validates seats by comparing `registered + pending` against `max_passengers`.
- Inserts into `bookings` with status = pending or registered (depends on rules).
- Updates counters in `flights` (pending_passengers / registered_passengers).
- If payment_type is `account`, deducts from passenger balance.

6.2 Cancel booking

- Sets booking status to cancelled.
- Decrements the correct passenger counter on `flights`.
- If payment_type was `account`, refunds the balance.

These operations should be transactional to avoid partial updates. Some implementations do use transactions; if not, race conditions can happen under load.

7) Endpoint: backend/api/cancel-flight.php

Purpose: company cancels/completes a flight and triggers refunds.

- Marks the flight as completed/cancelled (e.g., `is_completed=1`).
- Finds bookings that were paid via account and refunds passengers.
- Sets affected bookings to cancelled.

This endpoint is used from company dashboard actions.

8) Endpoint: backend/api/messages.php

Purpose: messaging between passenger and company, optionally linked to a flight.

Typical behaviors in this file:

- List conversations for a user (recent message per “other user”).
- Fetch messages between two users (optionally filtered by flight_id).
- Send a message (insert into `messages`).

9) “World B” endpoints in backend/api

Some endpoints like `airports.php`, `airlines.php`, `aircraft.php` are aligned with an airports/airlines schema (World B). They usually:

- Return reference data (list of airports, airlines, etc.) for dropdowns.
- Assume tables exist in the DB dump, not in `project_schema.sql`.

10) Files covered in Part 3

- `backend/api/get-cities.php`
- `backend/api/search-flights.php`
- `backend/api/company-flights.php`
- `backend/api/company-flight-details.php`
- `backend/api/passenger-bookings.php`
- `backend/api/bookings.php`
- `backend/api/cancel-flight.php`
- `backend/api/messages.php`

- `backend/api/airports.php`, `airlines.php`, `aircraft.php` (World B)

End of Part 3/5.